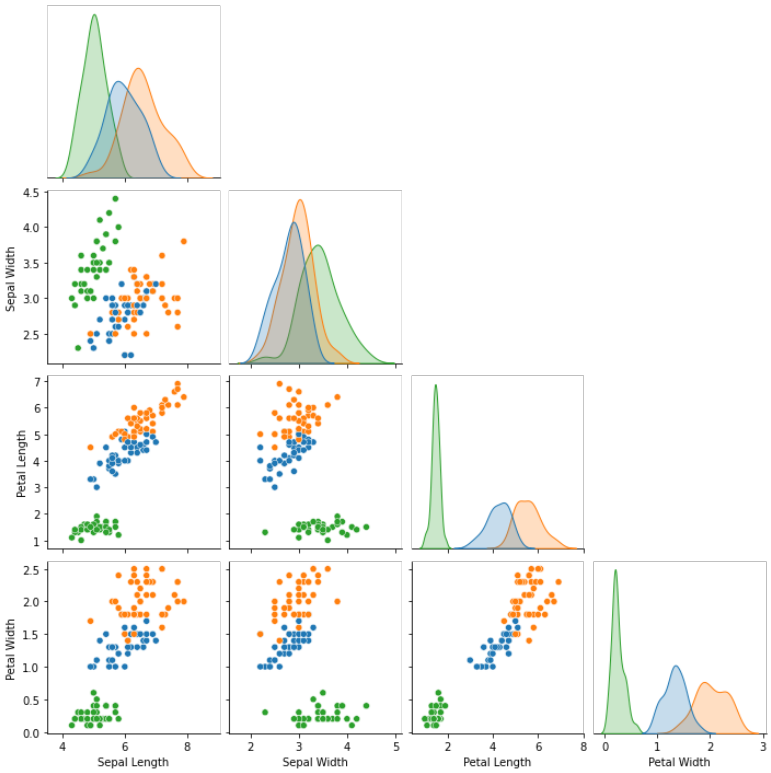
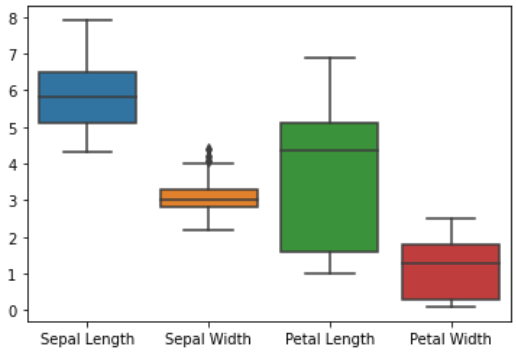
#Problem Statement: Dataset has 150 observations that contain Sepal Length, Sepal Width, Petal Length, Petal Width for 4 types of #flowers Setosa, Virginica, Versicolor. Build an algorithm in Python to classify a flower type based on its Sepal and Petal #dimensions.

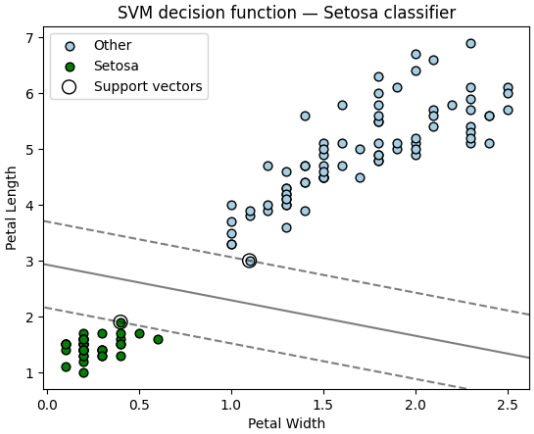
```
#Import and Clean the Dataset
import pandas as pd
url     = "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"
columns = ["Sepal Length", "Sepal Width", "Petal Length", "Petal Width", "Class"]
data = pd.read_csv(url, header=None, names=columns)
data = data.dropna(how="all").reset_index(drop=True)
classlabelmap = {"Iris-setosa" : "Setosa", "Iris-virginica" : "Virginica", "Iris-versicolor" : "Versicolor"}
train_columns = ["Sepal Length", "Sepal Width", "Petal Length", "Petal Width" ]
data["Class"] = data["Class"].apply(lambda x : classlabelmap.get(x))
classtypes = ["Setosa", "Virginica", "Versicolor"]
for classtype in classtypes :
    data[classtype] = data["Class"].apply(lambda x : x == classtype )

#Split data into train and test
from sklearn.model_selection import train_test_split
train, test = train_test_split(data, test_size = 0.2, random_state = 10)

#Visualize the Dataset
import seaborn as sns
import numpy as np
import matplotlib.pyplot as plt
plot_train = train.drop(classtypes,axis = 1)
plt.figure()
sns.boxplot(data = plot_train[train_columns])
sns.pairplot(plot_train, hue = "Class",corner = True )
```








```
#From the pair plot, it is evident that Petal Length and Petal Width clearly separate Setosa from other classes. Therefore, it
#makes sense to have a model that first checks whether a given training example is of type Setosa or not. Also note that the
#distribution of these attributes is approximately Gaussian which can be of use in classifying Non Setosa Iris plants.

#Fit an SVM model to classify Setosa Iris plants using Petal Length and Petal Width
from sklearn import svm
setosa_classifier = svm.SVC(kernel = "linear", C = 1000)
setosa_classifier.fit(train[["Petal Width", "Petal Length"]], train["Setosa"])
print("Accuracy of the SVM Setosa Classifier is ", setosa_classifier.score(train[["Petal Width", "Petal
Length"]],train["Setosa"])*100, "%" )

#Accuracy of the SVM Setosa Classifier is 100.0 %

def Plot_SVM(X, Y, Output, clf):
    import numpy as np, matplotlib.pyplot as plt
    m = np.asarray(Output, dtype=bool)
```

```
    fig, ax = plt.subplots()
    ax.scatter(X[~m], Y[~m], c=[plt.cm.Paired(0)], s=40, edgecolors='k', label='Other')
    ax.scatter(X[m],  Y[m],  c='green', s=40, edgecolors='k', label='Setosa')
    ax.set(xlabel='Petal Width', ylabel='Petal Length', title='SVM decision function – Setosa classifier')
    xlim, ylim = ax.get_xlim(), ax.get_ylim()
    xx = np.linspace(xlim[0], xlim[1], 200); yy = np.linspace(ylim[0], ylim[1], 200)
    XX, YY = np.meshgrid(xx, yy)

    #plot decision function contours
    Z = clf.decision_function(np.c_[XX.ravel(), YY.ravel()]).reshape(XX.shape)
    ax.contour(XX, YY, Z, colors='k', levels=[-1, 0, 1], alpha=0.5, linestyles=['--','-','--'])
    ax.scatter(clf.support_vectors_[:,0], clf.support_vectors_[:,1], s=100, linewidth=1,
               facecolors='none', edgecolors='k', label='Support vectors')
    ax.legend()
    plt.show()

Plot_SVM(train["Petal Width"], train["Petal Length"], train["Setosa"], setosa_classifier)

#Fit a Logistic Regression Model to classify Virginica and Versicolor
from sklearn.linear_model import LogisticRegression
train_nonsetosa = train[train["Setosa"] == False]
train_nonsetosa = train_nonsetosa.reset_index(drop = True)
virginica_predictor_lr = LogisticRegression()
virginica_predictor_lr.fit(train_nonsetosa[train_columns], train_nonsetosa["Virginica"])
print("Accuracy of the Logistic Regression Virginica Classifier is ",
virginica_predictor_lr.score(train_nonsetosa[train_columns],train_nonsetosa["Virginica"])*100, "%" )

#Accuracy of the Logistic Regression Virginica Classifier is 95.0 %

# Let's now fit a LDA model to classify Virginica from Versicolor
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
virginica_predictor_lda = LinearDiscriminantAnalysis()
virginica_predictor_lda.fit(train_nonsetosa[train_columns], train_nonsetosa["Virginica"])
print("Accuracy of the LDA Virginica Classifier is ",
virginica_predictor_lda.score(train_nonsetosa[train_columns],train_nonsetosa["Virginica"])*100, "%" )

#Accuracy of the LDA Virginica Classifier is 97.5 %

#Let's now fit a QDA model to classify Virginica from Versicolor
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
virginica_predictor_qda = QuadraticDiscriminantAnalysis()
virginica_predictor_qda.fit(train_nonsetosa[train_columns], train_nonsetosa["Virginica"])
print("Accuracy of the QDA Virginica Classifier is ",
virginica_predictor_qda.score(train_nonsetosa[train_columns],train_nonsetosa["Virginica"])*100, "%")

#Accuracy of the QDA Virginica Classifier is 96.25 %

#Compare all the 3 models for Classifying Virginica
from sklearn.model_selection import cross_val_score
virginica_models = [virginica_predictor_lr, virginica_predictor_lda, virginica_predictor_qda]
for model in virginica_models :
    score = cross_val_score(model, train_nonsetosa[train_columns], train_nonsetosa["Virginica"], cv = 10)
    score = np.mean(score)
    print("10-Fold Cross Validation Score for ", model, " is ", score*100, "%")

#10-Fold Cross Validation Score for LogisticRegression() is 95.0 %
#10-Fold Cross Validation Score for LinearDiscriminantAnalysis() is 96.25 %
#10-Fold Cross Validation Score for QuadraticDiscriminantAnalysis() is 93.75 %

#Build on the Cross-Validation analysis let’s pick LDA model for the final model
def IrisClassifier(X):
    X = X[train_columns]
    X["Setosa"]     = setosa_classifier.predict(X[["Petal Width", "Petal Length"]])
    X["Virginica"]  = virginica_predictor_lda.predict(X[train_columns])
    X["Versicolor"] = ~X["Virginica"]
    X["Class"] = np.select([X["Setosa"], X["Virginica"], X["Versicolor"]], classtypes, default = "Unkown")
    return X

#Check performance of the final model on train data
print(f"The accuracy on the training dataset is {np.mean(IrisClassifier(train[train_columns])['Class']==train['Class']) *
100:.2f}%")

#The accuracy on the training dataset is 98.34%

#Now check performance of the final model on Test Data
print(f"The accuracy on the test dataset is {np.mean(IrisClassifier(test[train_columns])['Class']==test['Class']) * 100:.2f}%")

#The accuracy on the test dataset is 96.67%
```